

Learning Capacity of a Perceptron

Cameron Ryan

Mathematical and Computational Problem Solving Methods for Physics

Northeastern University

April 25, 2026

Contents

1	Introduction	2
2	Learning Capacity	2
2.1	Determining Solvability Using <code>linprog</code>	3
2.2	Experimental Results	3
3	Robustness	4
3.1	Optimization Method	5
3.2	Gardner and Derrida's Prediction	6
3.3	Robustness Experiments	7
4	MNIST Dataset Experiment	7
4.1	Perceptron Learning Rule	7
4.2	Calculating Accuracy	7
5	Conclusion	8
	References	9
A	Code	9
A.1	Reproducing Results	9

1. Introduction

Perceptrons simple models of neurons which are extremely important in machine learning and theoretical neuroscience. First developed by Frank Rosenblatt in the 1950s, they simulate the mechanics of a single neuron. The single neuron will produce an output given a weighted aggregation of its inputs.

Consider a set of m binary N dimensional input vectors $x^\mu \in \{-1, +1\}^N$, with corresponding labels $y^\mu \in \{-1, +1\}$. The learning problem of a perceptron is to find a weight vector $(J_1, \dots, J_N) \in \mathbb{R}^N$ satisfying:

$$y^\mu \sum_{i=1}^N J_i x_i^\mu > 0 \quad \text{for all } \mu \in \{1, \dots, m\}$$

If such a weight vector exists that can satisfy this set of equations for a dichotomy $((x^1, \dots, x^\mu), (y^1, \dots, y^\mu))$ is solvable. This is commonly known to be equivalent to a hyperplane existing between the patterns of each class.

2. Learning Capacity

According to Cover's work in [1], the number of solvable dichotomies comprised of m patterns in an N dimensional binary space is:

$$C(m, N) = 2 \sum_{k=0}^{N-1} \binom{m-1}{k}$$

Therefore, given a randomly generated dichotomy, the probability of it being solvable can be represented as:

$$P_{\text{success}}(m) = \frac{\text{number of solvable dichotomies}}{\text{number of dichotomies}} = \frac{C(m, N)}{2^m}$$

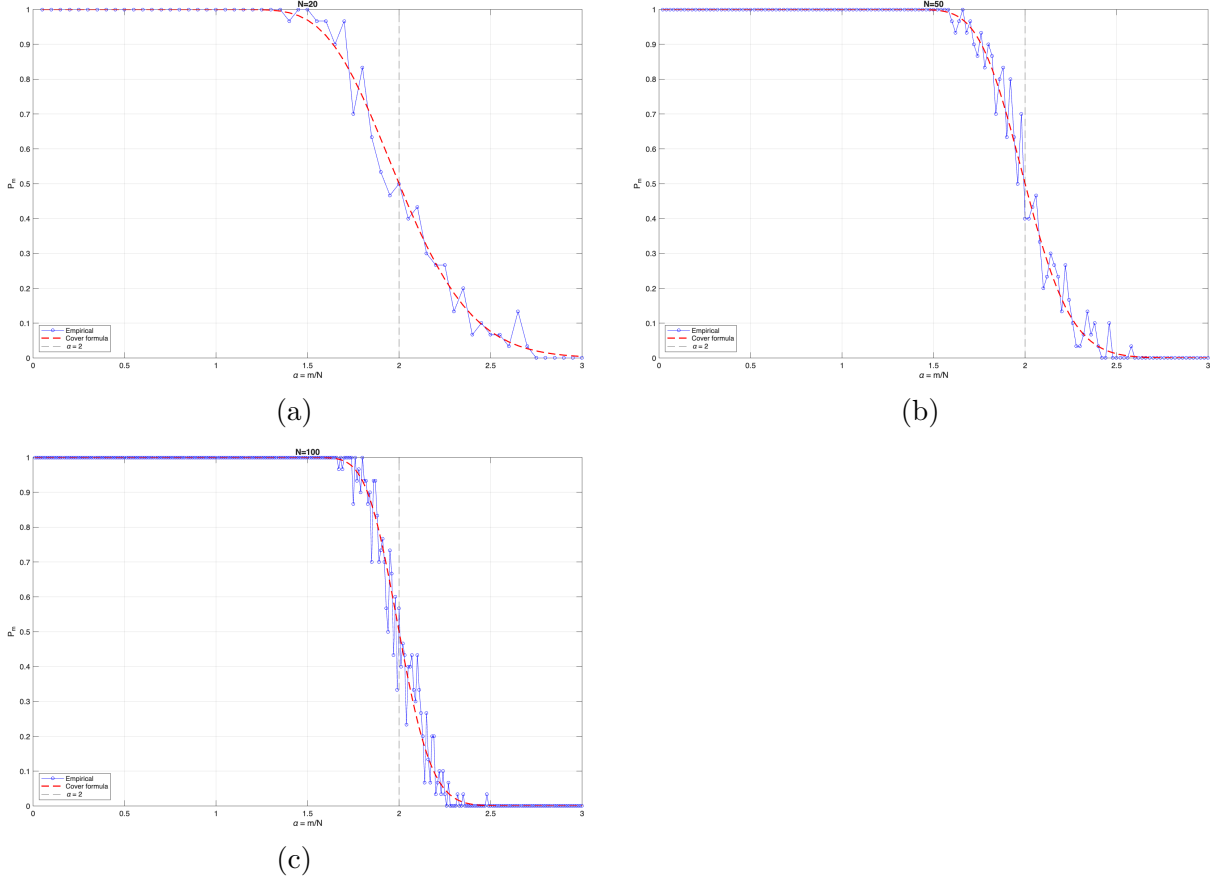


Figure 1: $P_{\text{success}}(m)$ experimentally tested for perceptrons of size (a) $N = 20$ inputs, (b) $N = 50$ inputs, and (c) $N = 100$ inputs plotted against the theoretical prediction by Cover [1].

2.1. Determining Solvability Using `linprog`

In order to determine whether a given dichotomy is solvable, we use MATLAB's `linprog` function. By setting f to be a zero vector, and A to be the matrix:

$$A = - \begin{pmatrix} y^1 x_1^1 & \dots & y^1 x_N^1 \\ \vdots & & \vdots \\ y^m x_1^m & \dots & y^m x_N^m \end{pmatrix}$$

Then using b as the zero vector, `linprog` will determine whether a solution to the inequality $A \cdot J < b$ exists.

2.2. Experimental Results

We ran experiments randomly sampling dichotomies for perceptrons of size $N = 20$, $N = 50$, and $N = 100$ and estimating the probabilities that they are solvable across

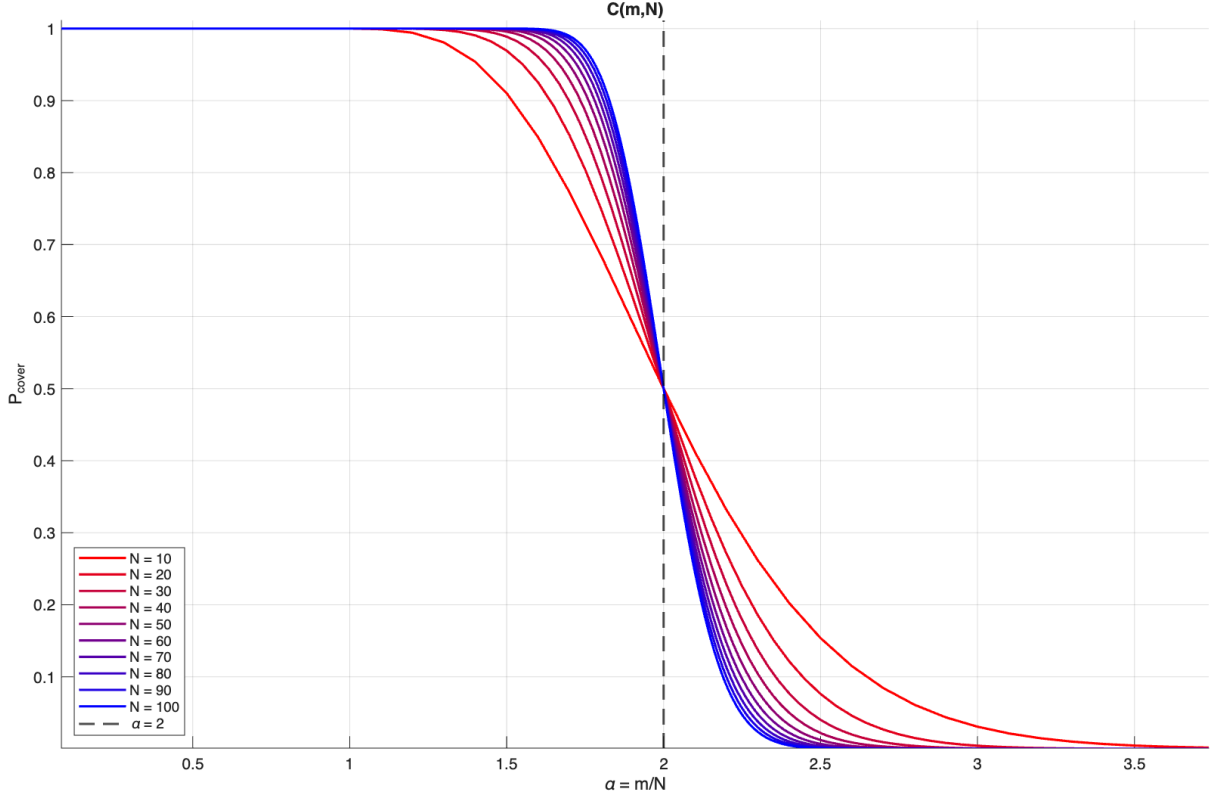


Figure 2: Cover's predicted $P_{\text{success}}(m)$ across varying N values. We see that as N increases, the probability behaves increasingly like a step function.

varying values of m . The results of this can be seen in Figure 1, which show a strong resemblance to Cover's theoretical probability.

We plot the probability of success as a function of $\alpha = m/N$. Visibly, we can observe a sharp decrease in learning capacity around $\alpha = 2$, which agrees with the findings of Gardner and Derrida in [2]. Interestingly, as N increases, the success probability $P_{\text{success}}(m)$ behaves more and more similarly like a step function, as can be seen in Figure 2.

3. Robustness

Another interesting analysis of the Perceptron learning capacity is by testing how robust the solution to a given dichotomy can be. Formally, instead of just determining whether a solution to equations exists, we wish to find the maximal κ such that:

$$y^\mu \sum_{i=1}^N J_i x_i^\mu \geq \kappa \quad \text{for all } \mu \in \{1, \dots, m\}$$

And in order to ensure the boundedness of this optimization problem, we constrain the Perceptron weights to be on the hypersphere of radius $\sqrt{N}$:

$$\sum_{i=1}^N J_i^2 = N$$

3.1. Optimization Method

In order to solve this using MATLAB's `quadprog`, we create an equivalent optimization problem. For convenience, define:

$$J' = \frac{J}{\kappa}$$

Therefore using the spherical constraint:

$$J \cdot J = N \tag{1}$$

$$\implies \kappa^2 J' \cdot J' = N \tag{2}$$

$$\implies \kappa = \sqrt{\frac{N}{J' \cdot J'}} \tag{3}$$

So now the problem of maximizing κ is equivalent to that of minimizing $J' \cdot J'$. Using `quadprog`, we can set A to be the same as before, and then setting:

$$H = 2I = \begin{pmatrix} 2 & \dots & 0 \\ \vdots & & \vdots \\ 0 & \dots & 2 \end{pmatrix}$$

We also set f to be a zero vector and b to be a one vector. Therefore `quadprog` will minimize:

$$\frac{1}{2} J'^T H J'$$

subject to the constraint $AJ' < b$. Once this has been solved, we can recover κ and the weight vector J .

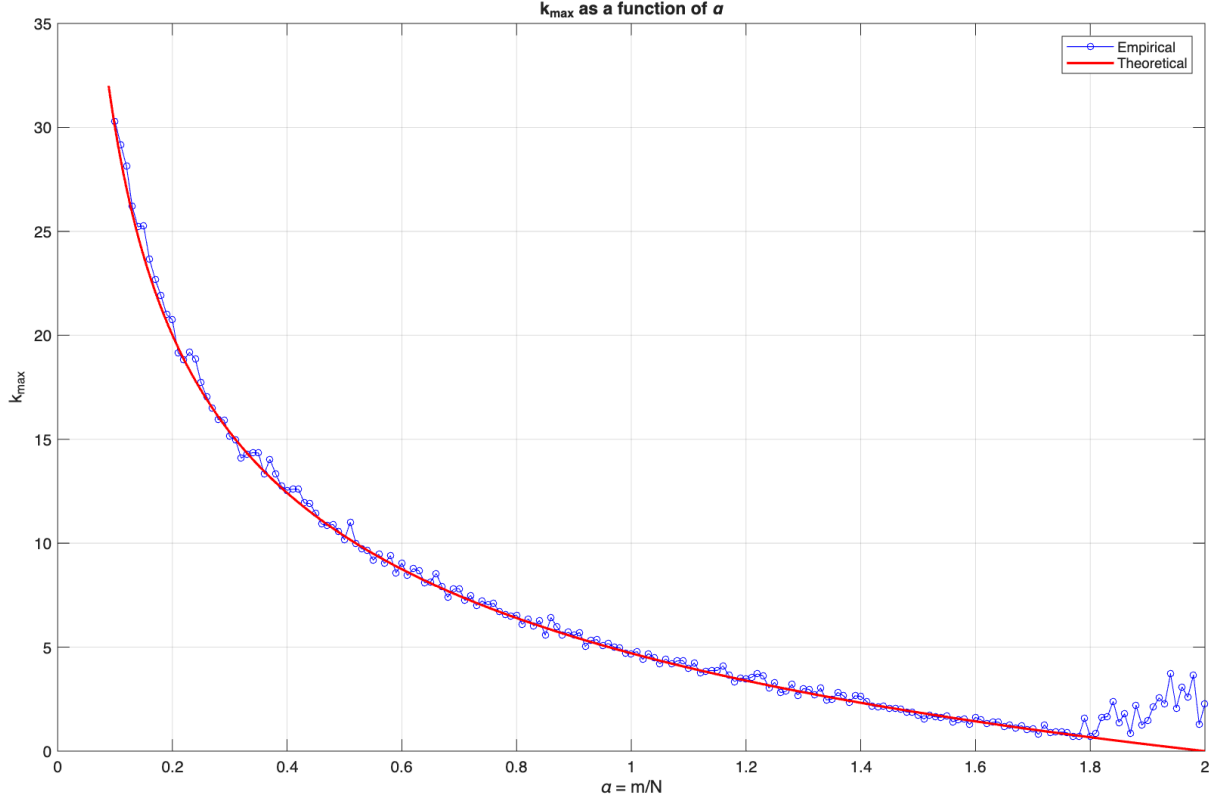


Figure 3: Simulated κ as function of $\alpha = m/N$ compared to the curve predicted by Cover [1].

3.2. Gardner and Derrida's Prediction

According to Gardner and Derrida [2], the relationship between $\alpha = m/N$ and the maximal robustness κ is represented as:

$$\alpha = \left(\int_{-\infty}^{\kappa/\sqrt{N}} \frac{dt}{(2\pi)^{1/2}} \exp(-t^2/2) (t - \kappa/\sqrt{N})^2 \right)^{-1}$$

Which can be solved analytically using Wolfram Alpha yielding:

$$\alpha = \left(\frac{1}{2} ((\kappa/\sqrt{N})^2 + 1) (\text{erf} \kappa/\sqrt{2N}) + \frac{\exp(-\kappa^2/2N) \kappa}{\sqrt{2\pi N}} \right)^{-1}$$

We plot this predicted curve in Figure 3. Empirical results show a strong alignment with this theoretical prediction.

3.3. Robustness Experiments

We performed simulations for a perceptron of size $N = 100$ across varying values of $\alpha = m/N$ in the range of $0 \leq \alpha \leq 2$. For each α , we determined the maximal margin $\kappa_{\max}$, and our results can be seen in Figure 3.

4. MNIST Dataset Experiment

The Modified National Institute of Standards and Technology (MNIST) dataset contains pixel grids for thousands of handwritten digits. Performing image classification on this dataset is frequently used in order to test the validity of machine learning methods. We trained a perceptron on each digit of the MNIST dataset (predict whether or not the image contains a particular digit or not).

4.1. Perceptron Learning Rule

We train the Perceptron using the perceptron learning rule. In order to do this, we will cycle through every sample in the dataset for several epochs. If the Perceptron correctly classifies a digit, then we will move on. If it incorrectly classifies a digit, then we update the weight according to the rule:

$$J_{t+1} \leftarrow J_t + y^\mu x^\mu$$

It is commonly known that if a dichotomy is solvable (linearly separable), then the perceptron will converge to the hyperplane in between the two classes of points.

4.2. Calculating Accuracy

In order to fairly judge the accuracy of the Perceptron in classifying a single digit, we must be careful due to the class imbalance (One label has 90% of the patterns, while another has only 10%). In order to get a better understanding of the competency of the Perceptron, we calculate the proportion of true positives and true negatives, then average those two values. This is what we consider to be our accuracy. Our final accuracies are recorded in Table 1 and Figure 4.

We observe that the accuracy of the perceptron on the train and test datasets appear to

be strongly correlated. Overall we get train and test accuracies of 90.44% and 89.91% respectively, which is good for many applications.

The accuracy scores achieved by the models appear to be highly dependent upon the digit. The digits 0 and 1 for example appear to be easily learnable by the perceptron, while others such as 5 seem to be more difficult. This is likely because it is difficult to discriminate between these digits. These digits have multiple ways that they are often written, meaning that the boundary between them and other digits is highly nonlinear.

Digit	Train Accuracy (%)	Test Accuracy (%)
0	97.30	97.60
1	98.61	98.54
2	87.74	85.55
3	92.14	92.68
4	94.09	93.41
5	77.81	78.16
6	96.92	96.54
7	94.64	94.01
8	74.96	72.73
9	90.22	89.86
Average	90.44	89.91

Table 1: Final train and test accuracy of the perceptron on each digit class of the MNIST dataset.

5. Conclusion

For this project, we experimentally verified previous research on the learning capacity of perceptrons. We found that the learning capacity for random dichotomies adheres to Cover’s rule [1], and that the maximally achievable robustness for a Perceptron scales as predicted by Gardner and Derrida [2]. Then we trained perceptrons to classify digits from the MNIST dataset. We found that the perceptron is effective at classifying some digits, while having trouble with others due to the nonlinear nature of their distributions.

References

- [1] T. M. Cover, “Geometrical and statistical properties of systems of linear inequalities with applications in pattern recognition,” *IEEE Transactions on Electronic Computers*, vol. EC-14, no. 3, pp. 326–334, 1965. DOI: 10.1109/PGEC.1965.264137.
- [2] E. Gardner and B. Derrida, “Optimal storage properties of neural network models,” *Journal of Physics A: Mathematical and General*, vol. 21, no. 1, pp. 271–284, 1988. DOI: 10.1088/0305-4470/21/1/031.

A. Code

The full source code for this project is available at:

<https://github.com/cryanwashere/Perceptron-Capacity-Math-Methods-Final->

The repository is structured as follows:

- `perceptron_1.m` — Code for part 1. Generates random dichotomies, and determines their solvability. Uses this to estimate $P_{\text{success}}(m)$, and compare it to Cover’s predictions.
- `perceptron_2.m` — Code for part 2. Calculates the maximal robustness at various values of α , and compares them to the curve predicted by Gardner and Derrida.
- `perceptron_3.m` — Code for part 3. Trains a perceptron on a specific target digit in the MNIST dataset, and graphs the train and test accuracy over epochs.
- `generate_dichotomy.m` — Generate a random dichotomy.
- `binarize_labels.m` — Convert MNIST labels to binary labels for a particular target digit.
- `cover_P.m` — Calculate analytical $P_{\text{success}}(m)$ predicted by Cover.
- `Cover_P_graph.m` — Generate graph for Figure 2.

A.1. Reproducing Results

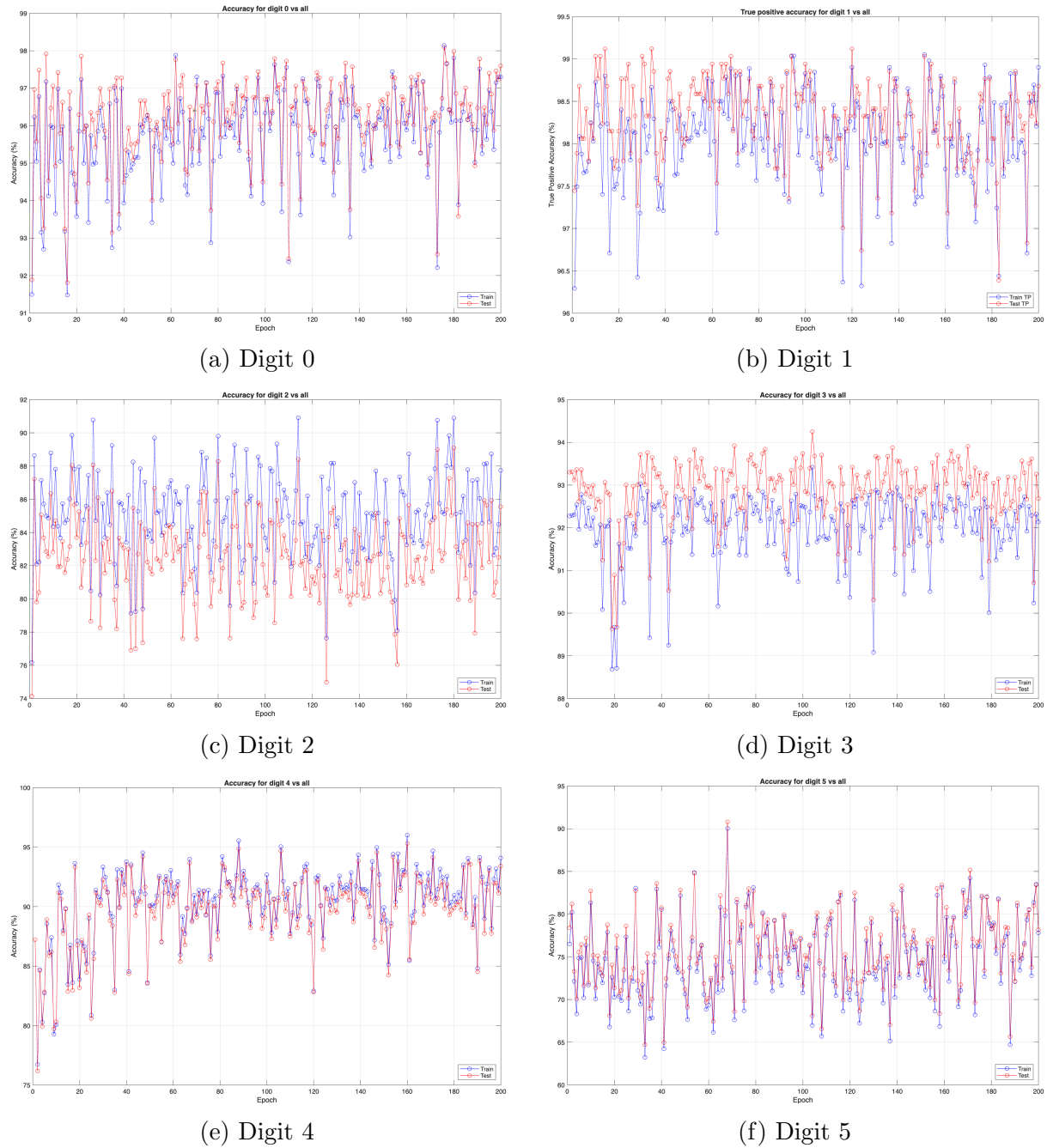
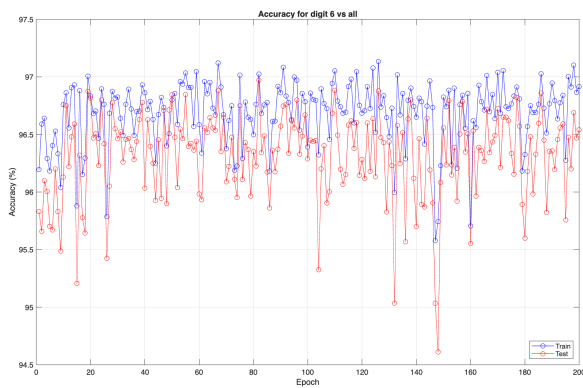
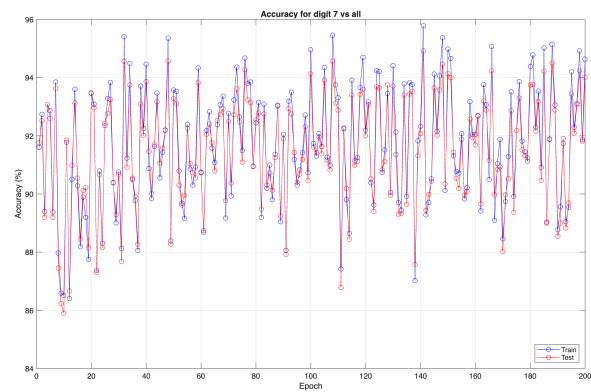


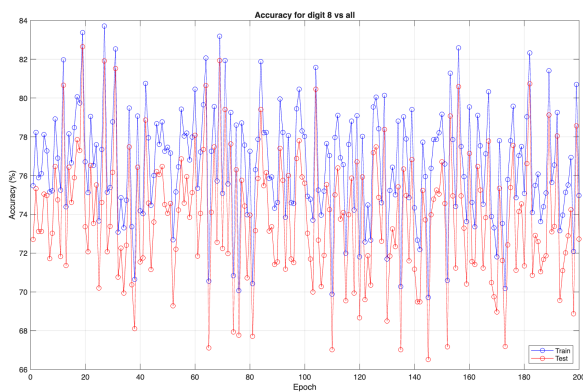
Figure 4: Training accuracy over epochs for a perceptron trained on each digit class of the MNIST dataset.



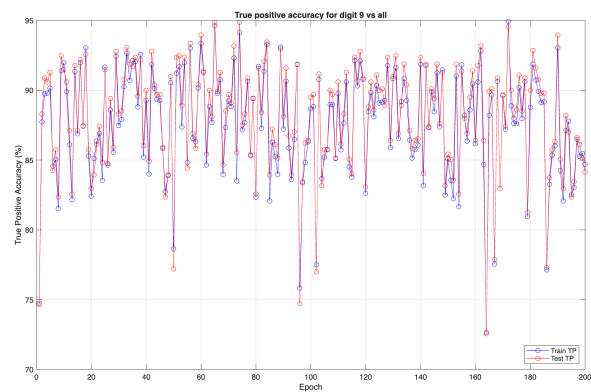
(g) Digit 6



(h) Digit 7



(i) Digit 8



(j) Digit 9

Figure 4: Training accuracy over epochs (continued).